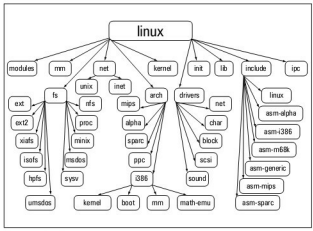

Figure 1: Linux in a tree structure.

- TCP/IP, SLIP and PPP support

*Though there are different mechanisms for efficient memory management, sometimes it may be eaten away. In that case, the OS looks for 4 KB memory pages that can be made free. Then the pages whose contents are already stored on the hard disk are identified and discarded.

The ability to support modules is really an amazing feature. In fact, there are few differences between normal programs and modules. In the case of normal C programs, they usually begin with a `main()` function and then execute a set of instructions. But in the case of modules, there will be an `init_module` or the function you point using the `module_init` call. This essentially speaks about the functionality that it can provide. And these modules end by calling `cleanup_module` (or like in the first case, the function you specify using a `module_exit` call).

We have seen that for a system call to perform, there should be a transition from user mode to the system mode. This is handled with the help of interrupts.

The parameters *sys_call_num* and *sys_call_args* are used to represent the number of the system call and its arguments. For example, we can have:

```
SAMPLE system_call( int sys_call_num , sys_call_args )
```

Let's look at some examples to comprehend this in a better way.

```
getpid (and getppid)
#include <sys/types.h>
#include <unistd.h>

pid_t
getpid(void);

pid_t
getppid(void);
```

The *getpid()* system call basically returns the process ID of the calling process. But it should not be used for constructing temporary file names due to security reasons.

For this purpose, we can use:

```
asmlinkage int sys_getpid(void)
{
return currcnt->pid;
}
```

Remember the *printk* we have used before? Beginners might find it hard to cope with new functions (which may appear undefined to them). To put it in simple words, I can say that there are functions available to modules. Here, too, you can find an included header file (more details about the subject can be accessed at en.wikipedia.org/wiki/Unistd.h).

And now let me describe a few commands (and their actions) for your reference. It will be useful when we proceed further:

- **kill** (send a signal to a process)

```
#include <sys/types.h>
#include <signal.h>

int
kill(pid_t pid, int sig);
```

- **init** (process control initialisation)

```
init
init [0 | 1 | 6 | c | q]
```

- telnetd

```
/usr/libexec/telnetd [-46BUhlkn] [-D debugmode] [-S tos] [-X
authtype]
 [-a authmode] [-edebug] [-p loginprog] [-u len]
 [-debug [port]]
```

- gethostname, sethostname

```
#include <unistd.h>

int
gethostname(char *name, size_t namelen);

int
sethostname(const char *name, int namelen);
```

We will continue to dedicate a few more days to review the literature. Happy kernel hacking!

By: Aasis Vinayak PG

The author is a hacker and a free software activist who does programming in the open source domain. He is the developer of V-language—a programming language that employs AI and ANN. His research work/publications are available at www.aasisvinayak.com